

Orquestación de agentes de lenguaje para el desarrollo de software

Una política de descomposición adaptativa con verificación basada en evidencia

Francisco Clarke
LU 120547

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

Directora: Dra. Ana Gabriela Maguitman
Codirector: Dr. Federico Martín Schmidt

2026

El problema

Un agente de lenguaje puede resolver una tarea de programación acotada. Un objetivo real de software no es una tarea acotada.

El problema

Un agente de lenguaje puede resolver una tarea de programación acotada. Un objetivo real de software no es una tarea acotada.

La salida natural es **descomponer**. Y ahí aparece la tensión:

Dividir de menos

- El agente satura su contexto.
- No hay puntos de verificación intermedios.
- Un fallo invalida todo el trabajo.

Dividir de más

- Aparecen contratos entre unidades.
- Crece el costo de integración.
- Los conflictos serializan el trabajo.

El problema

Un agente de lenguaje puede resolver una tarea de programación acotada. Un objetivo real de software no es una tarea acotada.

La salida natural es **descomponer**. Y ahí aparece la tensión:

Dividir de menos

- El agente satura su contexto.
- No hay puntos de verificación intermedios.
- Un fallo invalida todo el trabajo.

Dividir de más

- Aparecen contratos entre unidades.
- Crece el costo de integración.
- Los conflictos serializan el trabajo.

La paradoja de la granularidad: el óptimo no es un extremo, y no es el mismo para todas las tareas.

Preguntas de investigación

- RQ1 ¿Cómo varía la tasa de entrega verificada entre hoja única forzada, división fina fija y una política adaptativa?
- RQ2 ¿Qué trade-off existe entre éxito, duración, costo y overhead de coordinación?
- RQ3 ¿Qué modos de falla aparecen en cada configuración?

Estudio **exploratorio** de viabilidad y modos de falla.
No busca —ni podría alcanzar— significancia estadística.

La política de descomposición adaptativa

Cada unidad recibe cuatro señales normalizadas en $[0, 10]$:

$$C_{task} = w_1 S_r + w_2 I_i + w_3 V_s + w_4 T_m$$

Símbolo	Dimensión	Peso
S_r	Radio de alcance (archivos y módulos afectados)	0,30
I_i	Impacto sobre interfaces exportadas	0,25
V_s	Superficie de validación	0,25
T_m	Masa de contexto en tokens	0,20

Una unidad se mantiene como hoja si $C_{task} \leq 3,5$.

La política de descomposición adaptativa

Cada unidad recibe cuatro señales normalizadas en $[0, 10]$:

$$C_{task} = w_1 S_r + w_2 I_i + w_3 V_s + w_4 T_m$$

Símbolo	Dimensión	Peso
S_r	Radio de alcance (archivos y módulos afectados)	0,30
I_i	Impacto sobre interfaces exportadas	0,25
V_s	Superficie de validación	0,25
T_m	Masa de contexto en tokens	0,20

Una unidad se mantiene como hoja si $C_{task} \leq 3,5$.

Validación híbrida: el modelo *propone* las señales; un validador determinista las *acota* contra la superficie real del repositorio. El origen de cada señal queda registrado como *llm*, *clamped* o *derived*.

Dónde está la frontera

La política decide **si** dividir.

Solo el Arquitecto puede decir **cómo**.

Dónde está la frontera

La política decide **si** dividir.

Solo el Arquitecto puede decir **cómo**.

Esto no es una decisión de escritorio: es un **resultado negativo** obtenido de un run real.

- Una versión previa fabricaba sub-unidades particionando rutas de forma mecánica cuando el Arquitecto no proponía ninguna.
- En un run canónico los tres agentes terminaron con código de salida 0, y **los tres candidatos fueron rechazados por violar su propio contrato de alcance**.
- Causa: implementar una porción de la API también necesita el tipo del dominio del que depende. Una partición de rutas no es una unidad de trabajo coherente.

Dónde está la frontera

La política decide **si** dividir.

Solo el Arquitecto puede decir **cómo**.

Esto no es una decisión de escritorio: es un **resultado negativo** obtenido de un run real.

- Una versión previa fabricaba sub-unidades particionando rutas de forma mecánica cuando el Arquitecto no proponía ninguna.
- En un run canónico los tres agentes terminaron con código de salida 0, y **los tres candidatos fueron rechazados por violar su propio contrato de alcance**.
- Causa: implementar una porción de la API también necesita el tipo del dominio del que depende. Una partición de rutas no es una unidad de trabajo coherente.

Corrección: se retiró la síntesis mecánica. Sin sub-unidades propuestas se conserva la hoja cohesiva y se registra `resplit_declined`.

Recorrido de un run

1. **Grounding** — se indexa el repositorio objetivo; el plan cita rutas que existen.
2. **Planificación** — el planificador emite unidades semánticas y señales de complejidad.
3. **Política adaptativa** — C_{task} reforma el árbol; la decisión se persiste como evento.
4. **Compilación** — grafo con cuatro relaciones tipadas: pertenencia, artefactos, costuras y restricciones de conflicto.
5. **Ejecución** — cada hoja corre en un worktree de Git aislado, con *lease* y *fencing*.
6. **Validación** — matriz de evidencias sobre el commit exacto.
7. **Integración** — bottom-up, con reparación semántica.
8. **Entrega** — manifiesto y receipt sobre el árbol validado.

Dos invariantes que sostienen todo

1. **git diff es la única fuente de verdad.**

El agente no commitea. El orquestador inspecciona el árbol y commitea el diff exacto. Lo que el agente *dice* que hizo no entra en la evidencia.

2. **El alcance es un contrato, no una sugerencia.**

Cada nodo declara las rutas que puede modificar. Un archivo fuera de contrato invalida el candidato.

Dos invariantes que sostienen todo

1. `git diff` es la única fuente de verdad.

El agente no `commitea`. El orquestador inspecciona el árbol y `commitea` el `diff` exacto. Lo que el agente *dice* que hizo no entra en la evidencia.

2. El alcance es un contrato, no una sugerencia.

Cada nodo declara las rutas que puede modificar. Un archivo fuera de contrato invalida el candidato.

Este segundo invariante tuvo un costo medido: bajo alcance estricto, un archivo de prueba *nuevo* que el planificador no anticipó hacía fallar trabajo correcto. La corrección fue **creación acotada**: un nodo puede crear archivos bajo los directorios que ya posee —roots derivados por el compilador, nunca pedidos por el modelo—, pero editar un archivo preexistente no declarado sigue fuera de alcance.

Cómo se evaluó

Caso canónico: repositorio objetivo externo, commit base congelado, objetivo que cruza dominio, API, superficie web y pruebas. Codex CLI (gpt-5.5) en planificación y ejecución.

Estudio comparativo: 2 tareas $\times$ 3 condiciones $\times$ 2 repeticiones. Las dos tareas caen **en lados opuestos del umbral**: si ambas fueran complejas, la condición de hoja única fallaría por construcción y la política adaptativa parecería superior por una razón trivial.

Cómo se evaluó

Caso canónico: repositorio objetivo externo, commit base congelado, objetivo que cruza dominio, API, superficie web y pruebas. Codex CLI (gpt-5.5) en planificación y ejecución.

Estudio comparativo: 2 tareas $\times$ 3 condiciones $\times$ 2 repeticiones. Las dos tareas caen **en lados opuestos del umbral**: si ambas fueran complejas, la condición de hoja única fallaría por construcción y la política adaptativa parecería superior por una razón trivial.

Todo se fijó **antes** de observar resultados: tareas, orden, métricas, hipótesis, su falsador y una regla de inconclusión.

Las tablas y figuras se derivan automáticamente del journal de eventos. Ninguna cifra se transcribe a mano.

Lo que este diseño permite decir

Permite

- Caracterizar viabilidad.
- Describir trade-offs observados.
- Identificar **modos de falla**.
- Reportar la variabilidad del planificador.

No permite

- Afirmar superioridad de la política.
- Reportar significancia.
- Generalizar a otros repositorios.
- Calibrar los pesos.

Lo que este diseño permite decir

Permite

- Caracterizar viabilidad.
- Describir trade-offs observados.
- Identificar **modos de falla**.
- Reportar la variabilidad del planificador.

No permite

- Afirmar superioridad de la política.
- Reportar significancia.
- Generalizar a otros repositorios.
- Calibrar los pesos.

Con $n = 2$ por celda, una muestra chica rinde en lo cualitativo.
Los modos de falla son el resultado, no un subproducto.

Conclusiones

- Se construyó un orquestador que lleva un objetivo hasta un resultado **integrado, verificado y entregado**, con evidencia auditable de cada paso.
- La descomposición adaptativa está en la ruta productiva y su decisión queda **persistida y explicable** run por run.
- El hallazgo más informativo es negativo: una política determinista puede *detectar* exceso de complejidad, pero **no puede inventar el corte semántico**. Esa frontera es la contribución conceptual.
- Los defectos que importaron no se encontraron leyendo código, sino **ejecutando runs reales** contra un repositorio de verdad.

- Calibrar los pesos de C_{task} con una muestra que lo permita; el diseño actual, deliberadamente, no lo sostiene.
- Enmiendas versionadas de alcance: convertir una violación en una propuesta de revisión del grafo en lugar de una falla terminal.
- Ampliar el estudio a repositorios de dominios y tamaños distintos.
- Reducir la dependencia de la variabilidad del proveedor en planificación.

¿Preguntas?